

Proyecto de Evaluación: Arquitectura Políglota en «EcoDrive»

Víctor de Juan

Curso 2025-2026

1 Contexto del Proyecto: EcoDrive

La startup «**EcoDrive**» es una plataforma de movilidad urbana sostenible que ofrece alquiler de patinetes eléctricos, bicicletas y coches compartidos. Debido a su rápida expansión por toda Europa, el equipo de ingeniería se ha dado cuenta de que una única base de datos relacional tradicional ya no es capaz de soportar la diversidad y el volumen de sus datos.

Han decidido migrar a una arquitectura de **Persistencia Políglota**, apoyándose en tecnologías NoSQL. Tu misión como Arquitecto de Datos es diseñar, justificar e implementar los cuatro subsistemas críticos de la empresa.

Criterios de Evaluación (RA7)

- **CE.a:** Se han identificado las características de los sistemas NoSQL.
- **CE.b:** Se han evaluado los tipos de bases de datos NoSQL y su adecuación al caso de uso.
- **CE.c:** Se han identificado los elementos (agregados, claves, nodos, tipos de datos) utilizados en estas bases de datos.
- **CE.d:** Se ha gestionado la información utilizando comandos específicos.
- **CE.e:** Se han utilizado herramientas gráficas para la gestión de las bases de datos.

2 Fase 1: Análisis y Toma de Decisiones

A continuación, se describen los cuatro servicios que necesita EcoDrive. **Tu primera tarea es decidir qué tipo de base de datos NoSQL (Clave-Valor, Documental, Columnar o de Grafos) es la ideal para cada servicio y justificar tu elección.** Tienes a tu disposición Redis, MongoDB, Apache Cassandra y Neo4j en tu entorno Docker.

Servicio A: Catálogo de Vehículos y Mantenimiento

El catálogo contiene vehículos con características técnicas radicalmente distintas (un patinete tiene un límite de peso, una bicicleta tiene marchas y un coche tiene tipo de combustible). Además, se deben guardar historiales de reparación sin un esquema rígido, ya que las piezas a reparar varían constantemente.

Servicio B: Telemetría y Sensores IoT (Internet of Things)

Cada vehículo emite un «ping» cada 5 segundos indicando su ubicación exacta, velocidad y consumo de batería. Se generan millones de registros diarios. El equipo de Análisis de Datos necesita consultar rápidamente todo el historial de un vehículo concreto para detectar anomalías en la batería, por lo que las escrituras masivas sin bloqueos son críticas.

Pista de Diseño: *En este motor, el orden de los campos en la clave primaria determina cómo se reparten los datos en el clúster y cómo se ordenan en el disco. Consulta el **Anexo A1** para decidir qué campo garantiza que los datos de un mismo vehículo estén físicamente juntos.*

Servicio C: Sesiones Activas y Caché de Batería

La aplicación móvil necesita saber de forma casi instantánea qué vehículos están libres en un radio cercano y cuánta batería les queda. También se gestionan los tokens de sesión temporal de los usuarios que están actualmente realizando un viaje. Todo debe responder en microsegundos y los datos de sesión deben borrarse solos al terminar el viaje.

Servicio D: Red Social de «Carpooling»

EcoDrive quiere implementar un sistema para compartir coche. Necesitan recomendar compañeros de viaje basándose en si son «amigos», «amigos de amigos», o si suelen hacer rutas con puntos de origen y destino conectados.

3 Fase 2: Generación y Auditoría de Datos con IA

Una vez asignado un SGBD a cada servicio, deberás poblar las bases de datos. Sin embargo, en este módulo la IA suele cometer errores de «sesgo relacional». Tu misión no es solo generar código, sino auditarlo.

- Utiliza una IA (ChatGPT, Gemini, Claude...) para generar un **script de carga de datos inicial** para cada motor.
- **Auditoría Crítica:** Revisa minuciosamente las propuestas de la IA buscando los siguientes «Antipatrones NoSQL»:
 - **Normalización Encubierta (Mongo/Cassandra):** La IA tiende a separar datos en colecciones distintas y sugerir «joins manuales» por ID. Debes corregirlo usando **Agregados** (incrustar documentos) si los datos se leen siempre juntos.
 - **Diseño de Claves Erróneo (Cassandra):** Comprueba que la *Partition Key* no sea un ID secuencial genérico, sino un campo que permita distribuir los datos (ej. `vehicle_id`).
 - **Uso Pobre de Estructuras (Redis):** Evita que la IA use solo el tipo SET/GET (Strings) para todo. Audita si un HASH o un SORTED SET sería más eficiente para el caso de uso.
 - **Nodos-Tabla (Neo4j):** Asegúrate de que las relaciones tengan sentido semántico y dirección, y que no esté tratando los nodos como si fueran simples filas de una tabla SQL.

- En tu memoria, es **obligatorio** documentar al menos **dos correcciones técnicas** que hayas realizado sobre el código original de la IA, justificando por qué el cambio mejora la arquitectura del sistema.

4 Fase 3: Implementación y Consultas

Con los datos cargados, deberás ejecutar las siguientes operaciones en cada sistema.

Para el Sistema elegido en el Servicio A (Catálogo)

1. Inserta al menos 3 vehículos diferentes, aprovechando la flexibilidad del esquema.
2. Realiza una consulta que filtre los vehículos por un atributo que solo exista en uno de los tipos (ej. busca solo los que tengan «tipo de combustible»).

Para el Sistema elegido en el Servicio B (Telemetría)

1. Crea la estructura de tabla necesaria pensando primero en la consulta (*Query-First Design*). Define correctamente la *Partition Key*.
2. Inserta 5 registros de telemetría para un mismo vehículo en distintas horas.
3. Realiza una consulta para obtener el historial completo de telemetría de un vehículo específico.

Para el Sistema elegido en el Servicio C (Sesiones/Caché)

1. Crea un registro de sesión de viaje que expire automáticamente en 60 segundos.
2. Crea una estructura que guarde el nivel de batería actual de 3 vehículos distintos, y simula el consumo decrementando el valor de uno de ellos.
3. **Restricción de Nomenclatura:** Es obligatorio seguir la convención de jerarquía visual mediante dos puntos (ej: `sesion:token:ABC` o `vehiculo:id:bateria`), tal como se aplicó en el entorno GloboTour.

Para el Sistema elegido en el Servicio D (Red Social)

1. Inserta nodos y relaciones que simulen a 3 usuarios y 2 rutas/destinos.
2. Realiza una consulta que encuentre a los usuarios que viajan a un mismo destino y además son amigos entre sí.

5 Entregables

Deberás redactar una **Memoria Técnica en formato PDF** que contenga:

1. **Justificación Arquitectónica (CE.a, CE.b, CE.c):** Una tabla o listado explicando qué SGBD has elegido para cada Servicio. Debes identificar explícitamente los **elementos** de diseño: ¿Qué campos has incrustado en Mongo? ¿Qué tipo de datos de Redis has usado? ¿Cuál es tu *Partition Key* en Cassandra? ¿Qué etiquetas y relaciones usas en Neo4j?

2. **Análisis de Auditoría (CE.a, CE.c):** Bitácora de los cambios realizados sobre las propuestas de la IA, justificando técnicamente cada corrección basada en los principios del **anexo**.
3. **Scripts de Carga y Consultas (CE.d):** El código final (scripts de inserción y consultas de la Fase 3) utilizado en cada motor tras tu auditoría.
4. **Ejecución y Visualización (CE.e):** Capturas de pantalla demostrando la correcta ejecución de las consultas solicitadas en la Fase 3. Puedes usar tanto la consola de **DataGrip** como las interfaces visuales de cada contenedor (**RedisInsight**, **Mongo Express** o **Neo4j Browser**) donde se vea claramente el código introducido y el resultado devuelto.

Anexo: Guía Maestra de Auditoría y Modelado NoSQL

A1. El cambio de mentalidad en Cassandra: La Partition Key

En SQL, una **PRIMARY KEY** sirve para identificar una fila. En Cassandra, el diseño es **físico** y busca repartir el trabajo entre muchos servidores.

- **Partition Key (El «Dónde»):** Es el campo que Cassandra usa para decidir en qué servidor guarda el dato.
 - **Alta Cardinalidad (Deseable):** Necesitamos un campo que tenga muchos valores distintos posibles (ej. `vehicle_id` o `user_id`). Al haber miles de IDs diferentes, Cassandra puede repartir los datos de forma equilibrada por todo el clúster.
 - **Baja Cardinalidad (Antipatrón):** Si usas un campo con pocos valores posibles (ej. `tipo_vehiculo` o `estado_bateria`), Cassandra mandará todos los «Coches» al mismo servidor y todas las «Bicis» a otro. Si tienes 1 millón de coches, ese servidor explotará mientras los demás están vacíos. Esto se llama **Hotspot**.
- **Clustering Column (El «Orden»):** Decide el orden físico dentro del servidor. Si quieres consultar el historial de un vehículo de más reciente a más antiguo, el `timestamp` debe ser tu *Clustering Column*.
- **Query-First:** No diseñes tablas para «guardar cosas», diseñalas para «responder preguntas». Si necesitas dos formas de buscar, crea dos tablas con los mismos datos.

A2. MongoDB: ¿Incrustar o Referenciar? (RA7-CE.c)

- **Incrustar (Embedding):** Metes los datos relacionados dentro del mismo JSON. Es la opción por defecto. Úsalo si los datos se leen siempre juntos (ej. las piezas de un motor dentro del documento del coche).
- **Referenciar (Linking):** Guardas solo el ID. Úsalo solo si el dato «hijo» crece sin parar (millones de logs) o si se comparte entre muchos padres y cambia constantemente.

A3. Redis: Más allá del Texto (Key-Value avanzado)

- **HASHES:** No guardes un objeto como un JSON gigante. Usa HSET. Permite editar solo un campo (ej. bajar la batería un 1 %) sin tener que reescribir todo el coche en memoria.
- **TTL (Time To Live):** Fundamental para sesiones. Si la IA no usa EXPIRE, la memoria se llenará de sesiones de usuarios que ya se fueron hace días.

A4. Neo4j: Las Relaciones son Ciudadanos de Primera

- **Semántica:** Las relaciones deben tener dirección y nombre. No es lo mismo (User)-[:VIAJA_EN]->(Coche) que (Coche)-[:PERTENECE_A]->(User).
- **Propiedades en Bordes:** Puedes guardar datos en la propia flecha (ej. la distancia de una ruta o la fecha de una amistad).